

Request to Take Over Fundamental Locomotion Simulation Study

Hi Team,

I've spent some time studying two existing simulation projects that implement walking control for the Unitree Go2 robot using reinforcement learning agents:

1. Genesis Locomotion Example:

<https://github.com/Genesis-Embodied-AI/Genesis/tree/main/examples/locomotion>

2. Argo Robot Locomotion:

https://github.com/Argo-Robot/quadrupeds_locomotion/tree/main/src

Both use PPO-based training and follow a typical reinforcement learning structure with:

- An env file that defines the RL environment
- A train script for training the agent
- An eval script to load a trained agent and evaluate its performance by controlling the robot

Key Observations from My Study:

1. Environment Structure:

While the codebase loosely follows the Gym environment format, it significantly deviates from standard Gym conventions. Instead of using the typical `gym.Env` interface, it employs a tensor-based environment, which behaves differently. From what I can infer, this may be due to its origins—possibly developed initially for NVIDIA Isaac Sim and then modified for more efficient GPU utilization.

However, this non-standard structure makes the code harder to follow, debug, and extend, especially for tasks like adding new behaviors or integrating with other frameworks.

2. Custom RL Libraries:

The training framework does not rely on widely adopted libraries like Ray RLlib or Stable-Baselines. Instead, it uses custom-built RL components. While this may offer more control to the original developers, it becomes a bottleneck for us, particularly for maintainability and scalability. Reproducing or modifying the training process is significantly harder without standard tools and documentation.

3. CUDA Dependency:

I also ran into runtime issues when attempting to execute the simulation on systems without CUDA installed. This CUDA dependency creates a practical limitation for environments where GPU support isn't available or for quick experimentation on CPU-based setups.

What Needs to Be Done Next:

1. Successfully run the existing simulations without any errors, confirming that everything works as originally intended.
2. Understand each component—especially the observation space, action mapping, reward structure, and the internal training workflow.
3. Rebuild the simulation using the standard Gym interface, and switch to a standard RL library such as Ray RLlib. Once this is done, we'll have a cleaner and more extensible foundation to add features and adapt the locomotion to our system needs.

I'd like someone from the team to take over this simulation study so I can begin focusing on the next stage—developing the upper-level controller. I'll be happy to walk you through everything I've understood so far and provide any support you need to get started.

Let me know who's available and interested.

Best Regards,

Ritwik